

Complete guide to open source DevSecOps development

Code quality security compliance review and mandatory deployment validation

Edition 1.0 | 12 September 2026 | ERNU.EU

This manual explains how to implement a reproducible DevSecOps pipeline from scratch in a new GitHub project. Start with a manual non-blocking audit, calibrate findings, and then promote the controls into a mandatory gate for merge, release and deployment.

Intended readers: developers, GitHub administrators, DevOps engineers, security owners and technical managers.

Profile	Trigger	Effect
manual-nonblocking	workflow_dispatch	reports only
pr-advisory	pull request	annotations
mandatory-deployment-gate	PR / release	blocks deployment

Contents

1. Recommended decision
2. Architecture and responsibilities
3. Preparing the GitHub project
4. Installing SonarQube Community
5. Installing and configuring scanners
6. Manual non blocking operation
7. Interpreting findings and exceptions
8. Moving to mandatory validation
9. Protecting deployment
10. ORT compliance and SBOM
11. PR Agent with Ollama
12. Operations maintenance and troubleshooting
13. New project checklist
14. Complete file appendices

Recommended decision

Recommended stack

SonarQube Community, Semgrep CE, MegaLinter, Trivy, Gitleaks and Reviewdog form the recommended base. ORT adds license compliance and software inventory. PR-Agent with Ollama provides self-hosted AI review. No single tool covers quality, SAST, vulnerabilities, secrets, infrastructure as code and licenses.

Implementation principle

The initial manual-nonblocking profile runs only on request and publishes artifacts. After a two-to-four-week baseline, stable controls move to mandatory-deployment-gate. Keep the job name devsecops-gate stable for branch protection.

Architecture and responsibilities

Components and roles

SonarQube owns the quality gate; Semgrep detects insecure patterns; MegaLinter normalizes compliance; Trivy scans CVEs, IaC, filesystems and images; Gitleaks detects secrets; Reviewdog annotates diffs; ORT evaluates licenses; Ollama keeps AI review inside controlled infrastructure.

Control	Tool	Decision
Quality	SonarQube	Quality Gate
SAST	Semgrep	Rule severity
Compliance	MegaLinter	Lint policy
Supply chain	Trivy and ORT	CVE license SBOM
Secrets	Gitleaks	Any confirmed secret
Review	Reviewdog and PR-Agent	Human approval

Trust model

Grant the workflow only contents read by default. Add security-events write only when uploading SARIF. Keep tokens in GitHub Secrets or a secret manager. Do not expose self-hosted SonarQube or Ollama runners directly to the internet.

Preparing the GitHub project

Requirements

Use a GitHub repository with Actions enabled, Docker Engine and Compose v2. SonarQube should have at least 4 vCPU, 8 GiB RAM, persistent PostgreSQL, DNS, TLS and tested backups. Separate analysis runners from production hosts.

Area	Minimum	Production recommendation
Runner	Linux and Docker	Ephemeral or dedicated
SonarQube	4 vCPU 8 GiB	Capacity monitored
Database	PostgreSQL volume	Encrypted tested backup
Network	DNS and TLS	Private access and reverse proxy

Repository layout

Place policies in devsecops/config, helpers in devsecops/scripts and active workflows in .github/workflows. Files ending in .example remain inactive. Exclude generated reports from Git.

Installing SonarQube Community

Platform installation

Copy platform/sonarqube/.env.example to .env and replace every CHANGE_ME value. Validate docker compose config, start services and publish only the SonarQube UI through HTTPS. Do not expose PostgreSQL. Change the initial password and create a dedicated project token.

GitHub integration

Create SONAR_HOST_URL and SONAR_TOKEN as Actions Secrets. The scanner runner must resolve DNS and reach SonarQube. Never put real addresses or tokens into workflows or reports.

Installing and configuring scanners

Semgrep MegaLinter Trivy and Gitleaks

Keep policies under version control. During the pilot, commands publish SARIF even when findings exist. For blocking mode, remove exit-code 0 and continue-on-error, pin images and enable only calibrated rules.

Reviewdog

Reviewdog places findings on changed lines and reduces pull-request noise. Grant pull-requests write only to the annotation workflow. Scanner output remains the authoritative technical evidence.

Manual non blocking operation

Manual procedure

Validate configuration, then run the local helper or Actions Run workflow. Download artifacts, deduplicate and confirm findings, assign an owner and due date, remediate and rerun. The manual audit must not trigger deployment.

Classification

Confirmed secrets and critical vulnerabilities receive immediate priority. Scope false-positive suppressions narrowly. Every exception records the finding, component, rationale, compensating control, owner and expiry.

Interpreting findings and exceptions

Gate calibration

Measure runtime, stability, false-positive rate and backlog. Do not enable blocking while a scanner is unreliable. Start with confirmed secrets and critical CVEs, then add HIGH severity and agreed quality rules.

Branch protection

Enable Require a pull request before merging and Require status checks to pass. Select exactly devsecops-gate. Disable ordinary-contributor bypass, require an up-to-date branch, and protect deletion and force pushes.

Setting	Value	Purpose
Pull request	Required	No direct merge
Status check	devsecops-gate	Stable contract
Bypass	Disabled	Consistent enforcement
Environment	Required reviewers	Production approval

Moving to mandatory validation

Deployment contract

Build, package and deploy jobs declare needs on devsecops-gate. Deployment runs only after success. Scan the final image or artifact after build. Protect Production with GitHub Environment required reviewers.

Mandatory negative test

Add a harmless detected fixture in a test pull request. Confirm that the gate fails, merge is disabled and deployment does not start. Remove the fixture and confirm that the same pipeline becomes green.

Protecting deployment

ORT

ORT inventories packages, licenses and provenance, then evaluates organizational policy. Publish results and the SBOM as artifacts. An unknown license requires review; a denied license blocks the mandatory profile.

PR Agent and Ollama

Run Ollama on an internal host with runner-only access. Start PR-Agent in advisory mode. Do not send secrets or restricted code to external services. AI review never replaces human approval.

ORT compliance and SBOM

Maintenance

Update images and actions monthly through a dedicated pull request. Test SonarQube and PostgreSQL backup and restore. Monitor runtime, infrastructure failures and exception expiry. Distinguish scanner failure from a security finding.

Troubleshooting

If SonarQube is unreachable, verify DNS, TLS and runner routing. If SARIF upload fails, verify security-events write. If scans time out, exclude generated content and split jobs without reducing critical coverage.

New project checklist

- ☐ New repository contains no real organization data
- ☐ Secrets configured outside Git
- ☐ Manual workflow completed
- ☐ SARIF and SBOM artifacts available
- ☐ Baseline triaged and exceptions expire
- ☐ Stable devsecops-gate job
- ☐ Required status check enabled
- ☐ Build and deployment depend on gate
- ☐ Final image scanned
- ☐ Negative test demonstrated
- ☐ SonarQube backup and restore tested
- ☐ Owner and maintenance schedule approved

Complete file appendices

The appendices reproduce the files distributed with the toolkit. Pin actions and container images to approved tags or digests before production use.

Manual workflow

[templates/github/manual-devsecops-generic.yml](#)

```
name: Manual DevSecOps Audit

on:
  workflow_dispatch:
    inputs:
      scan_depth:
        description: Audit depth
        required: true
        default: standard
        type: choice
        options: [standard, extended]

permissions:
  contents: read

jobs:
  semgrep:
    runs-on: ubuntu-latest
    container: semgrep/semgrep:latest
    steps:
      - uses: actions/checkout@v4
      - run: semgrep scan --config auto --sarif --output semgrep.sarif .
        continue-on-error: true
      - uses: actions/upload-artifact@v4
        if: always()
        with: {name: semgrep-report, path: semgrep.sarif}

  trivy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          scanners: vuln,secret,misconfig
          severity: HIGH,CRITICAL
          format: sarif
          output: trivy.sarif
          exit-code: '0'
      - uses: actions/upload-artifact@v4
        if: always()
        with: {name: trivy-report, path: trivy.sarif}

  gitLeaks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: {fetch-depth: 0}
      - name: GitLeaks history audit
        run: docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect --source=/repo --report-
        format=sarif --report-path=/repo/gitleaks.sarif --exit-code=0
      - uses: actions/upload-artifact@v4
```

```
if: always()  
with: {name: gitleaks-report, path: gitleaks.sarif, retention-days: 14}
```

Mandatory gate example

[templates/github/mandatory-devsecops-gate.yml.example](https://github.com/ernu.eu/mandatory-devsecops-gate.yml.example)

```
name: Mandatory DevSecOps Gate
on:
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
  security-events: write
jobs:
  devsecops-gate:
    name: devsecops-gate
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: Gitleaks
        uses: gitleaks/gitleaks-action@v2
      - name: Trivy gate
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          severity: HIGH,CRITICAL
          exit-code: '1'
      - name: Semgrep gate
        run: docker run --rm -v "$PWD:/src" -w /src semgrep/semgrep:latest semgrep scan --config p/default
    --error .
```

Semgrep policy

[config/.semgrep.yml](#)

```
rules:
- id: ernu-no-shell-true-suppression
  message: Nu ascunde eșecul unui control de securitate cu `|| true`.
  severity: WARNING
  languages: [bash]
  patterns:
    - pattern-regex: '(semgrep|trivy|gitleaks).*\|\|\s*true'
- id: ernu-no-hardcoded-private-key
  message: Material de cheie privată detectat în cod.
  severity: ERROR
  languages: [generic]
  pattern-regex: '-----BEGIN (RSA |EC |OPENSSH )?PRIVATE KEY-----'
```

MegaLinter policy

[config/.mega-linter.yml](#)

```
APPLY_FIXES: none
VALIDATE_ALL_CODEBASE: true
SHOW_ELAPSED_TIME: true
PRINT_ALPACA: false
REPORT_OUTPUT_FOLDER: megalinter-reports
DISABLE_LINTERS:
  - REPOSITORY_TRIVY
  - REPOSITORY_GITLEAKS
MARKDOWN_MARKDOWNLINT_FILTER_REGEX_EXCLUDE: "(^|/)vendor/|^|/)node_modules/|^|/)artifacts/"
```

Gitleaks policy

[config/gitleaks.toml](#)

```
title = "ERNU DevSecOps Gitleaks policy"

[extend]
useDefault = true

[[allowlists]]
description = "Documented fake values and fixtures"
paths = [
    '''(^|/)\.env\.example$''',
    '''(^|/)\.fixtures?/''',
    '''(^|/)\.test-data/'''
]
regexes = [
    '''CHANGE_ME''',
    '''example\.invalid''',
    '''192\.0\.2\.[0-9]{1,3}'''
]
```

SonarQube Compose

[platform/sonarqube/docker-compose.yml](#)

```
services:
  sonarqube:
    image: sonarqube:community
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/${SONAR_DB_NAME}
      SONAR_JDBC_USERNAME: ${SONAR_DB_USER}
      SONAR_JDBC_PASSWORD: ${SONAR_DB_PASSWORD}
    ports:
      - "127.0.0.1:${SONAR_HTTP_PORT:-9000}:9000"
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs
    networks: [sonar]

  db:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${SONAR_DB_NAME}
      POSTGRES_USER: ${SONAR_DB_USER}
      POSTGRES_PASSWORD: ${SONAR_DB_PASSWORD}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${SONAR_DB_USER} -d ${SONAR_DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 10
    volumes:
      - sonarqube_db:/var/lib/postgresql/data
    networks: [sonar]

networks:
  sonar:
    internal: true

volumes:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:
  sonarqube_db:
```

SonarQube variables

[platform/sonarqube/.env.example](#)

```
SONAR_HTTP_PORT=9000  
SONAR_DB_NAME=sonarqube  
SONAR_DB_USER=sonarqube  
SONAR_DB_PASSWORD=CHANGE_ME_USE_A_LONG_RANDOM_PASSWORD
```

Validation script

[scripts/validate-configs.sh](#)

```
#!/usr/bin/env bash
set -Eeuo pipefail

root_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

python3 - <<'PY' "$root_dir"
import pathlib, sys
import yaml
root = pathlib.Path(sys.argv[1])
for path in root.rglob("*.yaml"):
    with path.open(encoding="utf-8") as handle:
        yaml.safe_load(handle)
    print(f"YAML OK: {path.relative_to(root)}")
PY

docker compose --env-file "$root_dir/platform/sonarqube/.env.example" \
-f "$root_dir/platform/sonarqube/docker-compose.yaml" config --quiet
echo "Compose OK"
```

Local audit script

[examples/sample-application/devsecops/scripts/run-local-audit.sh](#)

```
#!/usr/bin/env bash
set -Eeuo pipefail
repo_root="$(cd "$(dirname "${BASH_SOURCE[0]}")/../../.." && pwd)"
report_dir="${repo_root}/devsecops/reports"
mkdir -p "${report_dir}"
command -v docker >/dev/null || { echo "Docker este necesar." >&2; exit 2; }
docker run --rm -v "${repo_root}:/src" -w /src semgrep/semgrep:latest semgrep scan --config
/src/devsecops/config/.semgrep.yml --config p/default --sarif --output
/src/devsecops/reports/semgrep.sarif . || semgrep_status=$?
docker run --rm -v "${repo_root}:/src" aquasec/trivy:latest fs --scanners vuln,secret,misconfig --severity
HIGH,CRITICAL --format sarif --output /src/devsecops/reports/trivy.sarif /src
docker run --rm -v "${repo_root}:/src" zricethezav/gitleaks:latest detect --source=/src
--config=/src/devsecops/config/gitleaks.toml --report-format=sarif
--report-path=/src/devsecops/reports/gitleaks.sarif --exit-code=0
docker run --rm -v "${repo_root}:/tmp/lint" -e MEGALINTER_CONFIG=devsecops/config/.mega-linter.yml
oxsecurity/megalinter:v9
printf 'Rapoarte: %s; Semgrep status=%s\n' "${report_dir}" "${semgrep_status:-0}"
```

Project variables

[examples/sample-application/devsecops/config/project.env.example](#)

```
PROJECT_KEY=sample-application
SCAN_SEVERITY=HIGH,CRITICAL
SEMGREP_CONFIG=p/default
REPORT_DIR=devsecops/reports
SONAR_HOST_URL=https://sonarqube.example.invalid
SONAR_TOKEN=
```

Sonar project

[examples/sample-application/devsecops/sonar-project.properties](#)

```
sonar.projectKey=sample-application
sonar.projectName=Sample Application
sonar.sources=src
sonar.exclusions=**/vendor/**,**/node_modules/**,**/build/**
sonar.sourceEncoding=UTF-8
```